

Complexité algorithmique

September 21, 2026

1 Introduction

Avant de parler théorie, on part d'un exemple concret. Soit un annuaire téléphonique papier trié par ordre alphabétique, contenant 1000 noms et on cherche un nom précis. Deux algorithmes sont possibles :

- *Recherche séquentielle*

On part de la première page, et on regarde nom par nom jusqu'à trouver celui qu'on cherche (ou jusqu'à la fin si le nom n'existe pas).

```
[ ]: def recherche_séquentielle(annuaire, nom):  
    n = len(annuaire)  
    for i in range(n):  
        if annuaire[i] == nom:  
            return i  
    return -1 # non trouvé
```

Dans le meilleur des cas (le nom est en première position) on aura besoin d'une seule comparaison. Mais on s'intéresse au pire des cas (le nom est en dernière position, ou n'existe pas), dans ce cas on doit examiner tous les 1000 noms de l'annuaire.

- *Recherche binaire (dichotomie)*

Puisque l'annuaire est trié, on peut faire mieux : on ouvre le livre au milieu. Si le nom cherché vient avant alphabétiquement, on jette toute la seconde moitié et on recommence sur la première moitié — et inversement.

```
[ ]: def recherche_binaire(annuaire, nom):  
    gauche, droite = 0, len(annuaire) - 1  
    while gauche <= droite:  
        milieu = (gauche + droite) // 2  
        if annuaire[milieu] == nom:  
            return milieu  
        elif annuaire[milieu] < nom:  
            gauche = milieu + 1  
        else:  
            droite = milieu - 1  
    return -1 # non trouvé
```

À chaque étape, on élimine la moitié des possibilités restantes.

Pour bien saisir la différence on peut comparer graphiquement le nombre de comparaisons effectuées, au pire des cas, des deux algorithmes pour différentes tailles de l'annuaire :

```
[1]: import matplotlib.pyplot as plt  
  
def recherche_séquentielle(tab, x):  
    c = 0  
    for i in range(len(tab)):  
        c += 1
```

```

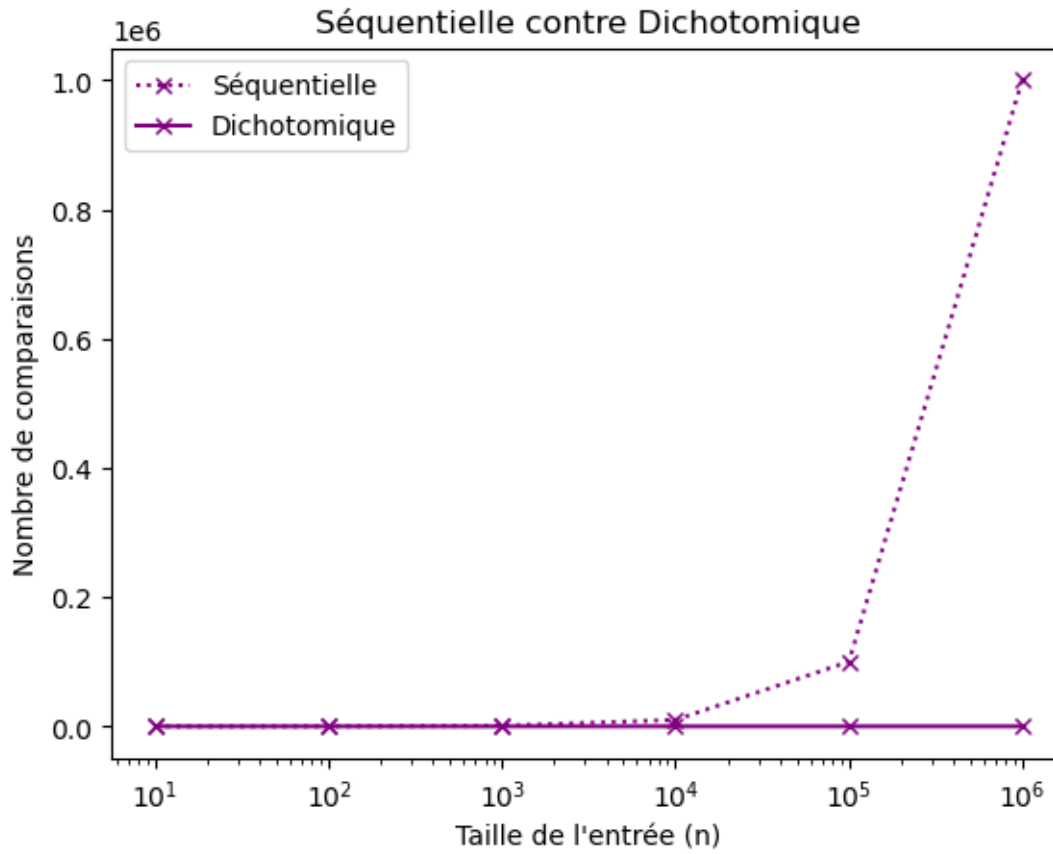
        if tab[i] == x:
            return c
    return c

def recherche_dichotomique(tab, x):
    c = 0
    gauche, droite = 0, len(tab) - 1
    while gauche <= droite:
        c += 1
        milieu = (gauche + droite) // 2
        if tab[milieu] == x:
            return c
        elif tab[milieu] < x:
            gauche = milieu + 1
        else:
            droite = milieu - 1
    return c

# Expérience : pire cas (élément absent) pour différentes tailles
tailles = [10, 100, 1000, 10000, 100000, 1000000]
c1 = []
c2 = []
for n in tailles:
    tab = list(range(n))          # liste triée 0, 1, ..., n-1
    x = -1                        # valeur absente -> force le pire cas
    c1.append(recherche_séquentielle(tab, x))
    c2.append(recherche_dichotomique(tab, x))

# Trace graphique
plt.plot(tailles, c1, marker='x', color='purple', linestyle=':', label='Séquentielle')
plt.plot(tailles, c2, marker='x', color='purple', label='Dichotomique')
plt.xscale('log')
plt.xlabel("Taille de l'entrée (n)")
plt.ylabel("Nombre de comparaisons")
plt.legend()
plt.title('Séquentielle contre Dichotomique')
plt.show()

```



On remarque facilement que la recherche séquentielle a une vitesse de croissance plus rapide que celle de la recherche dichotomique. Pour un million de noms, la recherche séquentielle peut demander un million de comparaisons, alors que la recherche binaire en demande à peine 19. C'est précisément ce genre d'écart que la théorie de la complexité permet de prévoir et de comparer, sans avoir à tester le programme sur des exemples géants.

Ce n'est pas la rapidité d'un ordinateur qui compte ici, mais la stratégie de l'algorithme. Un ordinateur 1000 fois plus rapide ne sauvera pas la recherche séquentielle face à la recherche binaire sur de très grands tableaux de données.

2 Qu'est-ce que la complexité ?

On distingue deux types de complexité :

- **Complexité temporelle** (mesure du temps) : La complexité temporelle d'un algorithme est une fonction, notée $T(n)$, qui donne le nombre d'opérations élémentaires exécutées par l'algorithme, en fonction de la taille n des données en entrée.
- **Complexité spatiale** (mesure de l'espace) : La complexité spatiale d'un algorithme est une fonction $S(n)$ (ou parfois notée $M(n)$) qui donne la quantité de mémoire supplémentaire (à l'exception des données d'entrée) utilisée par l'algorithme, en fonction de la taille n des

données en entrée.

Les deux mesures sont théoriques et sont faites indépendamment de n'importe quelle machine réelle.

3 Cas d'études

Pour une taille d'entrée n , on note E_n l'ensemble de toutes les entrées possibles de taille n , et pour une entrée $x \in E_n$ on note $T(x)$ le temps d'exécution de l'algorithme sur cette entrée précise. On distingue trois cas :

- **Pire cas** :

$$T_{\text{pire}}(n) = \max_{x \in E_n} T(x)$$

- **Cas moyen** :

$$T_{\text{moyen}}(n) = \sum_{x \in E_n} p(x) T(x)$$

où $p(x)$ est la probabilité de l'entrée x .

- **Meilleur cas** :

$$T_{\text{meilleur}}(n) = \min_{x \in E_n} T(x)$$

Généralement, on s'intéresse au premier cas, car il garantit que l'algorithme ne fera jamais pire que ça.

4 Les notations asymptotiques

Pour décrire le comportement d'un algorithme quand n tend vers l'infini on utilise trois notations :

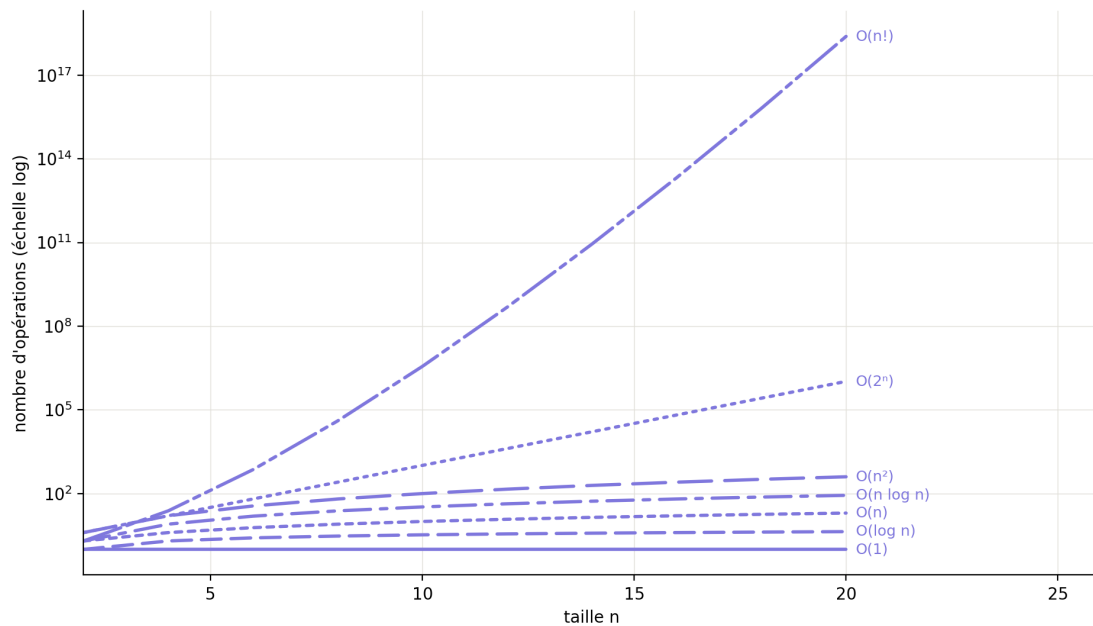
Notation	Définition formelle	Explication
$O(f(n))$	$\exists c > 0, n_0, \forall n \geq n_0, T(n) \leq c f(n)$	T ne grandit pas plus rapidement que f
$\Omega(f(n))$	$\exists c > 0, n_0, \forall n \geq n_0, c f(n) \leq T(n)$	T ne grandit pas plus lentement que f
$\Theta(f(n))$	$T(n) = O(f(n)) \text{ et } T(n) = \Omega(f(n))$	T et f ont le même ordre de grandeur

5 Les ordres de grandeur classiques

(de la plus lente à la plus rapide)

Notation	Nom
$O(1)$	Constant
$O(\log n)$	Logarithmique
$O(n)$	Linéaire
$O(n \log n)$	Quasi-linéaire
$O(n^2)$	Quadratique
$O(n^3)$	Cubique
$O(2^n)$	Exponentiel
$O(n!)$	Factoriel

et graphiquement :



Le graphe illustre bien une frontière fondamentale en complexité algorithmique entre deux familles d'algorithmes :

- **Courbes qui se tassent** : $O(1)$, $O(\log n)$, $O(n)$, $O(n \log n)$ et $O(n^2)$

Même pour de grandes valeurs de n , le temps de calcul reste praticable. Ce sont les complexités qu'on vise en général quand on conçoit un algorithme.

- **Courbes qui explosent** : $O(2^n)$ et $O(n!)$

Dès que la valeur de n devient un peu grande, le temps de calcul devient impraticable.

6 Exemple

On reprend l'exemple de la recherche séquentielle :

- **Pire cas** : L'élément recherché n'est pas présent, on parcourt tout le tableau en faisant n comparaisons, donc $T_{\text{pire}}(n) = O(n)$.

- **Meilleur cas** : L'élément est le premier dans le tableau, une seule comparaison est suffisante, donc $T_{\text{pire}}(n) = O(1)$.
- **Cas moyen** : $T_{\text{moyen}}(n) = \sum_{i=1}^n \frac{1}{n} i = \frac{1}{n} \frac{n(n+1)}{2} = O(n)$.

7 Calcul de complexité temporelle

7.1 Le modèle de référence : RAM

Le modèle **RAM** (Random Access Machine) est le modèle de référence standard pour l'analyse de complexité en algorithmique. C'est un modèle abstrait de calcul, indépendant de toute machine physique réelle.

Hypothèses du modèle :

- Mémoire adressable directement : accéder à n'importe quelle case mémoire se fait en temps constant $O(1)$, quel que soit l'endroit — d'où le nom "Random Access (Accès Aléatoire)".
- Instructions élémentaires à coût unitaire : addition, comparaison, affectation, lecture/écriture mémoire, affichage... coûtent chacune $O(1)$.
- Pas de parallélisme : une seule instruction exécutée à la fois (modèle séquentiel).

7.2 Démarche générale

- Identifier la taille des données n (longueur d'un tableau par exemple);
- Compter les opérations élémentaires en fonction de n ;
- Exprimer le coût total $T(n)$ comme une fonction de n ;
- Simplifier en ne gardant que l'ordre de grandeur dominant (notation O , Ω ou Θ).

7.3 Algorithme itératif

7.3.1 Séquence d'instructions

On additionne les coûts.

Exemple

```
[ ]: x = 0      #1 instruction (affectation)
     x += 1    #1 instruction (lecture + addition + affectation)
     print(x)  #1 instruction (affichage)
```

$$T = O(1) + O(1) + O(1) = O(1) \implies \text{Complexité constante}$$

.

Une suite d'instructions simples coûte toujours un temps constant, indépendant de toute variable n .

7.3.2 Structure répétitive

- **Boucle simple**

On multiplie le coût du corps (temps constant) de la boucle par le nombre d'itérations.

Exemple 1

```
[ ]: for i in range(n):    # répéter n fois
      print(i)           # 1 instruction élémentaire
```

$$T(n) = O(1) \times O(n) = O(n) \implies \text{Complexité linéaire}$$

Exemple 2

Une boucle qui divise (comme la recherche binaire)

On reprend la recherche binaire. À chaque tour de la boucle while, l'intervalle de recherche est divisé par 2. Le pire des cas est quand l'élément n'existe pas ou quand la taille de la liste devient 1. La question devient : combien de fois peut-on diviser n par 2 avant d'arriver à cette taille 1?

Soit k ce nombre de divisions, alors

$$\frac{n}{2^k} = 1 \implies k = \log_2 n$$

donc

$$T(n) = O(\log n) \implies \text{Complexité logarithmique}$$

- **Boucle imbriquée**

On multiplie le nombre d'itérations de la boucle externe par le nombre d'itérations de la boucle interne.

Exemple 1

```
[ ]: for i1 in range(n):
      for i2 in range(n):
          print(i1, i2)    # 1 opération
```

$$T(n) = O(n) \times O(n) = O(n^2) \implies \text{Complexité quadratique}$$

Exemple 2

```
[ ]: for i1 in range(n):
      for i2 in range(i1):
          print(i1, i2)
```

Le nombre total d'itérations est

$$0 + 1 + 2 + \dots + n - 1 = \frac{n(n-1)}{2}$$

donc $T(n) = O(n^2)$. Ce qui montre que même avec des bornes variables, on retombe sur du quadratique.

7.3.3 Structure conditionnelle

```
[ ]: if condition:
      # branche 1
else:
      # branche 2
```

On ne compte pas la somme des deux branches, une seule branche s'exécute selon la condition — jamais les deux. On ne peut pas savoir à l'avance laquelle, donc pour le pire cas on prend la pire branche :

$$T_{pire} = \max(T_{branche1}, T_{branche2})$$

Exemple

```
[ ]: if x == 0:
      for _ in range(n):      # O(n)
          print('ko')
else:
      print('ok')             # O(1)
```

La condition peut tomber dans la branche coûteuse, donc $T(n) = O(n)$

7.4 Algorithme récursif

Pour un algorithme récursif le coût d'un appel dépend du coût d'appels plus petits; on ne peut pas compter directement, il faut établir une relation de récurrence puis la résoudre.

Une récurrence exprime le temps d'exécution $T(n)$ d'un problème de taille n en fonction du temps d'exécution de sous-problèmes plus petits.

Exemple : La factorielle

```
[ ]: def factorielle(n):
      if n == 0:
          return 1
      return n * factorielle(n - 1)
```

- *Cas de base* ($n = 0$) : coût constant $O(1)$;
- *Cas général* : un coût constant (le test et la multiplication) plus le coût de l'appel récursif sur $n - 1$. On traduit ça par une relation de récurrence :

$$T(n) = \begin{cases} O(1) & \text{si } n = 0 \\ T(n-1) + O(1) & \text{sinon} \end{cases}$$

Il faut maintenant résoudre cette récurrence pour obtenir $T(n)$ en fonction de n seul, sans référence à T . Plusieurs méthodes classiques existent : la substitution, le théorème maître, et d'autres approches complémentaires.

7.4.1 Méthode de substitution

On devine (ou on calcule pas à pas) une forme close, puis on vérifie par récurrence (au sens mathématique) que cette forme satisfait bien l'équation.

Exemple 1 : La factorielle

- *Étape 1 — Déplier la récurrence*

Pour $T(n) = T(n-1) + c$ (avec c une constante) :

$$T(n) = T(n-1) + c = T(n-2) + 2c = T(n-3) + 3c = \dots = T(n-k) + kc$$

On s'arrête lorsqu'on atteint le cas de base (l'entrée égale 0), ce qui veut dire $n-k=0$ donc $k=n$.

$$\text{D'où } T(n) = T(0) + nc = O(1) + O(n) = O(n)$$

- *Étape 2 — Vérifier par récurrence (induction)*

On pose l'hypothèse $T(n) = nc + T(0)$ et on vérifie :

$$\text{Base : } T(0) = 0c + T(0) = T(0)$$

$$\text{Hérédité : Si } T(n-1) = (n-1)c + T(0), \text{ alors } T(n) = T(n-1) + c = (n-1)c + T(0) + c = nc + T(0)$$

Donc

$$T(n) = O(n)$$

Exemple 2 : Recherche dichotomique

```
[ ]: def recherche_dichotomique_récurrent(tab, x, début, fin):
    if début > fin:
        return False
    milieu = (début + fin) // 2
    if tab[milieu] == x:
        return True
    elif tab[milieu] < x:
        return recherche_dichotomique_récurrent(tab, x, milieu + 1, fin)
    else:
        return recherche_dichotomique_récurrent(tab, x, début, milieu - 1)
```

- *Étape 1*

$$T(n) = T(n/2) + c = T(n/4) + 2c = T(n/8) + 3c = \dots = T(\frac{n}{2^d}) + dc$$

On s'arrête lorsqu'on atteint le cas de base (la taille du tableau égale 1), ce qui veut dire $\frac{n}{2^d} = 1$ donc $d = \log_2 n$.

$$\text{D'où } T(n) = T(1) + c \log_2 n = O(1) + O(\log n) = O(\log n)$$

- *Étape 2 : Récurrence sur d*

On pose l'hypothèse $T(2^d) = T(1) + dc$ et on vérifie :

$$\text{Base : } T(2^0) = T(1)$$

$$\text{Hérédité : si } T(2^d) = T(1) + dc, \text{ alors } T(2^{d+1}) = T(\frac{2^{d+1}}{2}) + c = T(2^d) + c = T(1) + (d+1)c$$

Donc

$$T(n) = O(\log n)$$

La méthode de substitution est puissante mais demande de deviner la forme de la solution — ce qui devient difficile pour des récurrences plus complexes. C'est là qu'intervient le théorème maître.

7.4.2 Théorème maître

Ce théorème s'applique à des récurrences de la forme :

$$T(n) = aT\left(\frac{n}{b}\right) + f(n)$$

avec :

- a : le nombre de sous-problèmes (appels récursifs)
- b : le facteur de réduction de la taille du problème
- $f(n)$: le coût du travail fait en dehors des appels récursifs (instructions élémentaires, division du problème, ...)

On trouve trois cas en comparant $f(n)$ à $n^{\log_b a}$:

Cas	Condition	Conclusion
1	$f(n) = O(n^{\log_b a - \varepsilon})$ pour un $\varepsilon > 0$ (f croît strictement plus lentement)	$T(n) = \Theta(n^{\log_b a})$
2	$f(n) = \Theta(n^{\log_b a})$ (Même ordre de grandeur)	$T(n) = \Theta(n^{\log_b a} \log n)$
3	$f(n) = \Omega(n^{\log_b a + \varepsilon})$ pour un $\varepsilon > 0$ (f croît strictement plus rapidement)	$T(n) = \Theta(f(n))$

Le terme $(n^{\log_b a})$ correspond au nombre de sous-problèmes de taille 1 générés au total. Donc on compare qui "domine"; le coût de la récursion pure ($n^{\log_b a}$) ou le coût du travail à chaque niveau ($f(n)$). Le plus grand des deux l'emporte (cas 1 ou 3) ; s'ils sont égaux, on ajoute un facteur $\log n$. C'est une question de qui détermine la complexité finale.

Exemple 1 : Recherche dichotomique

On a $T(n) = T(n/2) + O(1)$

$a = 1$ et $b = 2$, donc $n^{\log_b a} = n^{\log_2 1} = n^0 = 1 = O(1)$

et

$$f(n) = O(1) = O(n^0)$$

Les deux ont le même ordre donc on tombe dans le cas 2 : $T(n) = \Theta(n^{\log_b a} \log n) = \Theta(\log n)$

Exemple 2 : Karatsuba - Multiplication de grands entiers (n représente le nombre de chiffres)

$$T(n) = 3T(n/2) + O(n)$$

$a = 3$ et $b = 2$, donc $n^{\log_b a} = n^{\log_2 3} = n^{1,585}$

et

$$f(n) = O(n)$$

Alors le travail récursif domine, d'où $T(n) = \Theta(n^{1,585})$, tandis que la multiplication naïve coûte $\Theta(n^2)$.

8 Calcul de complexité spatiale

Pour une exécution donnée sur une entrée de taille n , on note $M(n, t)$ le nombre de cases mémoire élémentaires occupée par l'algorithme à l'instant t de son exécution, sans compter l'espace occupé par les données d'entrée elles-mêmes.

On définit alors la complexité spatiale de l'algorithme, notée $S(n)$, comme :

$$S(n) = \max_t (M(n, t))$$

C'est-à-dire : la mémoire maximale utilisée à un instant donné, sur l'ensemble de l'exécution, pour une entrée de taille n .

Une case mémoire est une unité d'espace abstraite de taille constante $O(1)$, capable de stocker une donnée de base comme un nombre, un caractère, ...

8.1 Algorithme itératif

Exemple 1

```
[2]: def somme(liste):  
    s = 0  
    for x in liste:  
        s += x  
    return s
```

À chaque instant de l'exécution, il n'y a que la variable s en mémoire occupant une seule case élémentaire (en plus de $liste$, qu'on ne compte pas car c'est l'entrée). Donc $S(n) = O(1)$.

Exemple 2

```
[3]: def doubler(liste):  
    d = []  
    for x in liste:  
        d.append(2*x)  
    return d
```

Ici, d grandit progressivement. Le pic mémoire n'est pas atteint au début, mais à la toute fin, quand d contient les n éléments simultanément en mémoire. Donc $S(n) = O(n)$.

8.2 Algorithme récursif

Pour un algorithme récursif, $S(n)$ est liée à la profondeur maximale de récursion, car chaque appel actif occupe un espace constant sur la pile tant qu'il n'est pas terminé.

Si $P(n)$ est la profondeur maximale de récursion (nombre d'appels simultanément empilés au pic), alors :

$$S(n) = O(P(n))$$

Exemple

```
[1]: def factorielle(n):  
      if n == 0:  
          return 1  
      return n * factorielle(n - 1)
```

Chaque appel reste en attente dans la pile d'exécution tant que l'appel suivant n'est pas terminé, car il attend le résultat pour faire la multiplication une opération en $O(1)$. Le pic mémoire est atteint au moment où tous les appels intermédiaires sont simultanément en mémoire, empilés, en attente; C'est lorsqu'on atteint le cas de base après n appels. Donc $S(n) = O(1) \times O(n) = O(n)$.